

Design Document: Agentic Enzyme Engineering Planner

System Overview

The Enzyme Engineering Planner is an agentic AI system designed to automate the iterative process of rational protein design. The core motivation behind this system is that enzyme engineering is inherently non-linear; a researcher does not simply analyze a sequence once and arrive at a perfect mutation. Instead, they cycle through analysis, hypothesis, testing, and revision, often many times before arriving at a viable design. Traditional software pipelines fail to capture this iterative, judgment-driven process. An agentic architecture, by contrast, is uniquely suited to it.

The system is built around a **Blackboard Architecture**, in which a central shared state object acts as the "memory" of the entire workflow. Six specialist agents read from and write to this shared state, each contributing a different layer of biological expertise. A Planner agent sits at the top of this hierarchy, evaluating the current state of the project at each step and deciding which specialist should act next. The workflow is governed by a Generate-Critique-Revise loop, meaning that proposed mutations are never accepted immediately; they must pass through a skeptical Critic agent before a final report is issued. This design ensures that the system is not just fast, but scientifically responsible.

Agent Roles and Responsibilities

The system consists of six distinct agents, each with a clearly defined domain of expertise and a specific role in the workflow.

The **Planner Agent** serves as the orchestrator of the entire system. It does not perform any biological analysis itself. Instead, it reads the current state of the blackboard: what has been analyzed, what is missing, what the Critic has flagged, and then decides which agent should act next. It is the only agent with a global view of the workflow, and it is responsible for enforcing the stopping conditions of the loop. Its input is the full global state, and its output is a directive: a named agent and a set of task instructions.

The **Sequence Analysis Agent** is a computational biologist. Given a raw amino acid sequence, it performs statistical and conservation analysis, identifying regions of the protein that are likely to be structurally or functionally important. It uses ESM-2 embeddings to generate a "hotspot map", a ranked list of residue positions that are candidates for mutation. Its output is a structured set of conservation scores and flagged regions, which are written to the blackboard for downstream agents to use.

The **Functional Annotation Agent** is a domain expert in protein function. It queries the InterPro database to identify known functional domains, motifs, and active site residues within the input sequence. This is a critical safety layer in the system—by identifying which residues are catalytically essential, it provides hard constraints that the Mutation Designer must respect. Its output is a list of annotated domains and a set of "do not mutate" residue positions.

The **Structure Agent** is a structural biologist. It takes the raw amino acid sequence and submits it to ESMFold to generate a predicted 3D structure in PDB format. It then analyzes this structure to identify secondary structure elements (helices, sheets, loops), surface-exposed versus core-buried residues, and regions of structural flexibility or instability. This spatial information is essential for the Mutation Designer, who needs to know not just which residues are important, but where they are in three-dimensional space.

The **Mutation Designer Agent** is the creative engine of the system. Drawing on the outputs of all three analysis agents: sequence hotspots, functional constraints, and structural geometry, it proposes a set of specific point mutations in standard notation (e.g., A23V, K55R). For each mutation, it provides a short biological rationale, explaining why the substitution is expected to improve the target property (thermostability, solubility, or activity) without disrupting function. Its output is a structured list of mutation candidates written to the blackboard.

The **Critic Agent** is a skeptical quality control reviewer. It evaluates the Mutation Designer's proposals against a set of biological safety rules; for example, it will reject any mutation that touches a known active site residue, destabilizes the protein core, or lacks a clear justification. Its output is binary: either APPROVED, meaning the design is ready for reporting, or REJECTED, accompanied by specific feedback that the Planner will use to constrain the next redesign iteration.

Finally, the **Reporter Agent** is activated only when the loop terminates. It synthesizes the entire workflow, the analysis results, the proposed mutations, and the Critic's final verdict into a structured scientific summary that is rendered in the Flask user interface.

Coordination and Communication

All inter-agent communication in this system flows through the shared blackboard state, implemented as a Python dictionary. Agents do not call each other directly. Instead, each agent reads the fields it needs from the state, performs its task, and writes its results back. This design has two important advantages. First, it decouples the agents from each other, meaning any agent can be upgraded or replaced without affecting the others. Second, it gives the Planner a complete, up-to-date picture of the system at every decision point.

The coordination mechanism is a non-linear loop with a defined stopping condition. After each agent completes its task, control returns to the Planner, which re-evaluates the state and decides the next step. This means the workflow is not a fixed pipeline — if the Critic rejects a design, the Planner does not simply restart from the beginning. Instead, it reads the Critic's feedback, updates the constraints on the blackboard, and re-dispatches only the Mutation Designer with the new constraints. This targeted iteration is what makes the system genuinely agentic rather than just a sequential script.

The loop terminates under two conditions: either the Critic issues an APPROVED verdict, or the system reaches its maximum iteration count (default: 3). In the latter case, the Reporter Agent is still triggered, but its summary will reflect the fact that no approved design was found, giving the human user actionable information about why the sequence was difficult to engineer.

Failure Handling and Quality Control

Agentic systems are vulnerable to compounding errors: a bad output from one agent can corrupt the inputs of every downstream agent. This system addresses that risk through three layers of defense.

The first and most important layer is the Critic Loop. Every mutation proposal must pass through the Critic before it is accepted. If the Mutation Designer produces a biologically unsafe suggestion, the Critic catches it, explains why it was rejected, and the Planner uses that explanation as a hard constraint in the next iteration. This self-correcting loop is the system's primary quality-control mechanism.

The second layer is tool-level validation. Before any agent writes a result to the blackboard, it checks that the result is structurally valid. For example, the Structure Agent verifies that the ESMFold API returned a file with a valid PDB header before saving it. If an API call fails or returns malformed data, the agent

writes an error status to the blackboard, and the Planner routes around it, either retrying or skipping that analysis step.

The third layer is iteration capping. In cases where a sequence is genuinely difficult to engineer, because all candidate mutation sites are functionally constrained, the system will not loop indefinitely. The hard iteration cap ensures that the user always receives a response, even if that response is a well-documented explanation of why no viable design was found. This is an important design principle: a system that fails gracefully and informatively is far more useful to a researcher than one that either crashes or silently produces bad output.